# CS 636: Analysis of Concurrent Programs
# Project Report

Khushboo Agrawal (242110605)
Ashutosh Kumar (210221)

April 21, 2025

## 1 Problem Statement

Implementation of on-the-fly AeroDrome analysis (Atomicity Checking in Linear Time using Vector Clock) using Intel Pin

## 2 Solution

### 2.1 Assumptions

1) We have declared global variables and AeroDrome analysis tracks addresses of these variables only to check for conflict serializability.
2) Our pintool instruments pthread_mutex_lock function and pthread_mutex_unlock to invoke acquire_lock and release_lock routine of Aerodrome analysis
3) Our intel pintool instruments the function txn() which is a transaction for the purpose of Aerodrome analysis.

### 2.2 Approach

We use intel pintool to instrument all the routine in our benchmark program. If the routine encountered is main then the start of analysis is signaled. The global_vars function enables us to track the address of global variables for checking conflict serializability. If the name of the function instrumented by intel pin is thrd, then the fork and join rountine in Aerodrome analysis is invoked. The begin and end routine of Aerodrome analysis is invoked when the routine instrumented by pin is txn. We use intel pin tool APIs to track read and write to global variables, get the thread_id and parent_id.

### 2.3 Algorithm

Algorithm 1 represents driver code implemented for Aerodrome analysis to check for conflict serializability. The core of the algorithm iterates through all the routine in the benchmark program for instrumentation using intel pintool. For each routine in the benchmark program, we insert an instrumentation before and invoke checkmain function. If the routine is main then we set start to true. If the routine is global_vars, we set global to true. For each routine in the benchmark program, we insert an instrumentation before and invoke begin_func function. If the start and global are set to true then if routine is write_then we add address of global variables in the var_map and lock_map. If global is false and routine is pthread_mutex_lock then we invoke acquire routine of aerodrome analysis. If the function is thrd then we invoke fork function of the aerodrome analysis. If the routine is pthread_mutex_unlock then we invoke release lock routine of aerodrome analysis. If the routine is txn then we invoke begin function of aerodrome analysis. For each instruction in the routine, we check if the instruction is a write or read instruction using API of intel pin.

If the write and read is to address of global variables we invoke write and read functions in aerodrome analysis. For each routine we insert call after return and invoke retfunc. If the start is true and func is thrd we invoke join function of aerodrome analysis. If the routine is txn then we invoke end function of aerodrome analysis.

## 2.4 Challenges

The challenges faced in implementation of Aerodrome analysis are as follows:

- Finding the address of global variables and global locks for the purpose of AeroDrome analysis. To overcome this we have declared a function global_vars which inturn calls write function in which the global variables are initialised.This enables us to extract the address of the global variables. This function also calls pthread mutex lock which enables us to extract the address of the global locks
- Finding the parent id of the threads. To overcome this we have used the API in pintool. We used thrd prefix functions to indicate a new thread has been spawned and used this calling to find the new thread-id and parent thread-id
- we have defined a function txn() in our benchmark program,to demarcate transactions as there are not atomic blocks as in Java

## 2.5 Benchmark Schedule

In this section, we discuss the schedule in benchmark programs on which we demonstrate our Aerodrome Analysis. In the schedule given below W1(z) would mean write on variable z in transaction 1. Similarly r4(x) would mean read of variable x in transaction 4

### 2.5.1 Benchmark1

lockacq1(l),W1(x), W1(x) W1(x), lockrel1(l), lockacq2(l), W2(x), W2(x),W2(x), lockrel2(l)
output: Serializable

### 2.5.2 Benchmark2

W2(x), W1(x), W2(y), W3(x), W1(y), W3(y)
output: conflict serializable violation

### 2.5.3 Benchmark3

W1(x), W2(x), W3(y), W1(y)
output: Serializable

### 2.5.4 Benchmark4

W1(x), W2(y), W1(y), W2(x)
output: conflict serializable violation

### 2.5.5 Benchmark5

r4(x), r2(x), r3(x), r1(y), W1(y), W2(x) ,W3(y), W4(y)
output: Serializable

### 2.5.6 Benchmark6

W1(x), W2(x), W2(y), W1(y)
output: conflict serializable violation

**Algorithm 1** Driver for AeroDrome

```
1: function DRIVER_AERODROME(program)
2:     var_map ← []; lock_map = [ ]; start= false; global = false;
3:     while all_routines_instrumented == false do
4:         for each routine in RTN do
5:             RTN_InsertCall(checkmain, before)
6:             if routine == main() then
7:                 start = true
8:             end if
9:             if routine == global_vars() then
10:                global = true
11:            end if
12:        end for
13:        for each routine in RTN do
14:            RTN_InsertCall(begin_func, before)
15:            if routine == write_() then
16:                var_addr.insert(addr);
17:            else if routine == pthread_mutex_lock() then
18:                lock_addr.insert(addr);
19:            else if routine == thrd() then
20:                var_map[Addr] = val
21:                fork(threadid,parentid)
22:            else if routine == txn() then
23:                begin(threadid)
24:            else if routine == pthread_mutex_lock() then
25:                acquire(lockid)
26:            else if routine == pthread_mutex_unlock() then
27:                release(lockid)
28:            end if
29:        end for
30:        for each instruction in routine do
31:            INS_InsertCall(Instruction, before)
32:        end for
33:        for each routine in RTN do
34:            RTN_InsertCall(retfunc, After)
35:            if start == true then
36:                join(threadid, parentid);
37:            else if routine == txn() then
38:                end(thrdid)
39:            end if
40:        end for
41:    end while
42: end function
```

#### 2.5.7 Benchmark7

R1(x), R3(y), R3(x), R2(y), R2(z), W3(y),W2(z), R1(z), W1(x), W1(z)
output: Serializable

#### 2.5.8 Benchmark8

W2(y), W3(y), W1(z), W2(z),W3(x), W1(x)
output: conflict serializable violation

#### 2.5.9 Benchmark9

R1(x), R2(y), R3(y), W2(y),W1(x), W3(x), R2(x), W2(x)
output: Serializable

### 2.6 Command to run the code

- Create a directory 'Aerodrome' in source/tools/ directory of intel pintool
- Download the zipfile and extract the files in the submission folder and copy them in directory Aerodrome
- To create binary for benchmark program navigate to Aerodrome folder and run gcc -o test{*} test{*}.c where {*} is the testcase number
- To instrument the binary and run the Aerodrome analysis run make
- pin -t /Aerodrome/obj-intel64/submission.so – /Aerodrome/test{*}

### 2.7 Member Contribution

Equal contribution of both the member.